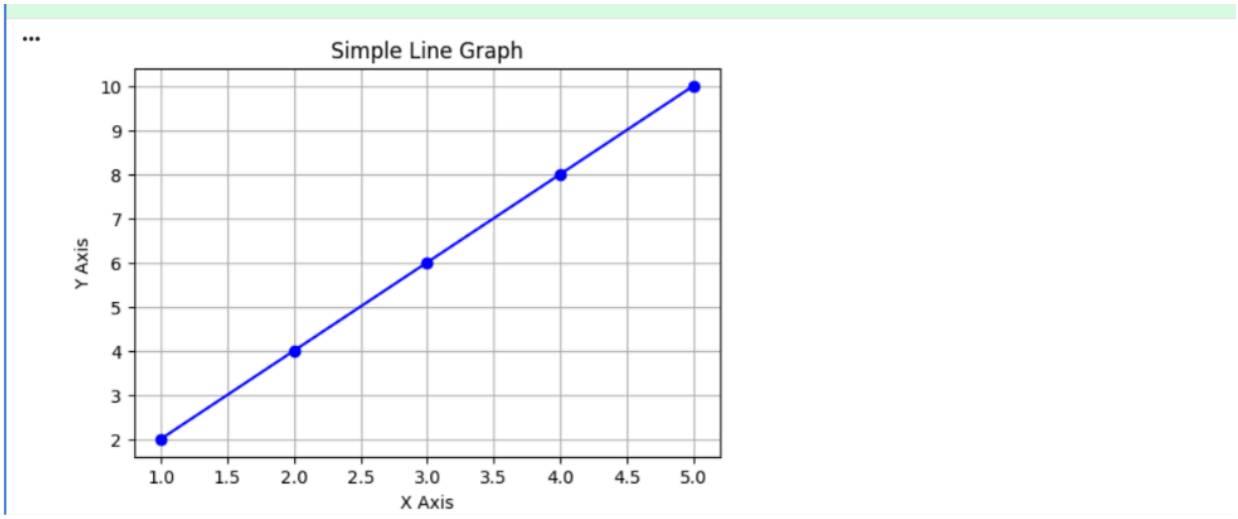
Course Outcome 1 Practical Exercises

1. Install Matplotlib and display a simple line graph.



2. Install Scikit-learn and print its version.

```
%pip install -q scikit-learn

import sklearn
print(f"Scikit-learn version: {sklearn.__version__}")
```

```
Scikit-learn version: 1.6.1
```

3. Create a NumPy array containing the first 20 natural numbers.

```
import numpy as np

natural_numbers = np.arange(1, 21)
print("First 20 natural numbers:", natural_numbers)
```

```
First 20 natural numbers: [ 1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20]
```

4. Create a Pandas DataFrame with student names and grades.

```
import pandas as pd

data = {
    'Student Name': ['Alice', 'Bob', 'Charlie', 'David', 'Eva'],
    'Grade': ['A', 'B', 'A+', 'B+', 'A-']
}

df = pd.DataFrame(data)
display(df)
```

	Student Name	Grade
0	Alice	A
1	Bob	B
2	Charlie	A+
3	David	B+
4	Eva	A-

5. Verify the installed versions of all AI libraries on your system.

```
libs = ['numpy', 'pandas', 'torch', 'transformers', 'tensorflow', 'scipy']

for lib in libs:
    try:
        imported_lib = __import__(lib)
        print(f"{lib}: {imported_lib.__version__}")
    except ImportError:
        print(f"{lib}: Not Installed")
```

```
numpy: 2.1.3
pandas: 2.2.3
torch: 2.11.0+cpu
transformers: 5.18.0
tensorflow: 2.21.0
scipy: 1.16.3
```

6. Create tensors using PyTorch and perform basic mathematical operations.

```
import torch

tensor1 = torch.tensor([5.0, 10.0, 15.0])
tensor2 = torch.tensor([2.0, 2.0, 2.0])

print("Addition:", tensor1 + tensor2)
print("Subtraction:", tensor1 - tensor2)
print("Multiplication:", tensor1 * tensor2)
print("Division:", tensor1 / tensor2)
```

```
Addition: tensor([ 7., 12., 17.])
Subtraction: tensor([ 3.,  8., 13.])
Multiplication: tensor([10., 20., 30.])
Division: tensor([2.5000, 5.0000, 7.5000])
```

7. Load a Pre-trained Transformer Model using Hugging Face and perform sentiment
Analysis.

```
%pip install -q transformers

from transformers import pipeline

classifier = pipeline('sentiment-analysis', model='distilbert-base-uncased-finetuned-sst-2-english')
result = classifier("I absolutely love learning state-of-the-art AI technologies!")
print(result)
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
config.json: 100% 629/629 [00:00<00:00, 5.30kB/s]
model.safetensors: reconstructing file: 100% 268MB / 268MB, 23.2MB/s
model.safetensors: downloading bytes: 250MB, 20.4MB/s
Loading weights: 100% 104/104 [00:00<00:00, 939.64it/s]
tokenizer_config.json: 100% 48.0/48.0 [00:00<00:00, 693B/s]
vocab.txt: 100% 232k/232k [00:00<00:00, 2.28MB/s]
[{'label': 'POSITIVE', 'score': 0.9998550415039062}]
```

8. Tokenization using BERT Tokenizer

```
from transformers import BertTokenizer

tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
text = "Tokenization converts text into clean individual tokens."
tokens = tokenizer.tokenize(text)

print("Tokens:", tokens)
```

```
tokenizer_config.json: 100% 48.0/48.0 [00:00<00:00, 1.83kB/s]
vocab.txt: 100% 232k/232k [00:00<00:00, 3.97MB/s]
tokenizer.json: 100% 466k/466k [00:00<00:00, 9.42MB/s]
Tokens: ['token', '##ization', 'converts', 'text', 'into', 'clean', 'individual', 'token', '##s', '.']
```

9. Generate text using a pre-trained GPT-2 model.

```
tokenizer.json: 100% 1.36M/1.36M [00:00<00:00, 23.0MB/s]
[transformers] Passing `generation_config` together with generation-related arguments=({'max_length', 'num_return_sequences'}) is deprecated and will be removed in future ve
[transformers] Both `max_new_tokens` (=256) and `max_length`(=30) seem to have been set. `max_new_tokens` will take precedence. Please refer to the documentation for more in
[transformers] Ignoring clean_up_tokenization_spaces=True for BPE tokenizer GPT2Tokenizer. The clean_up_tokenization post-processing step is designed for WordPiece tokenizer
Artificial Intelligence will reshape the way we live, work, and think about the future.

Now, one of the things that's been missing from the last few years is a framework that will be able to provide the necessary tools to make the world a better place.

That brings us to the next stage. For decades, the world has been thinking about how to do this. We have been thinking about what we can do now to make the world an even bet

That's where the next era of AI will take us. For a long time, those goals didn't seem to be going to be realized. The world was looking at a world where robots were going t

But today, we are getting there.

Today, we're getting there with the promise of future technology.

That's why I'm going to share with you the story of how I got started. I'm talking about a tech startup called Hacking the Future. It's our first ever meeting.

I'm talking about a startup called Hacking the Future .

We're going to get started by talking to a tech startup.

The thing that I'm going to be talking about is a company called Hacking the Future.
```

10. Load a pre-trained transformer model using the Hugging Face pipeline() API and perform
sentiment analysis.

```
sentiment_pipeline = pipeline('sentiment-analysis')
results = sentiment_pipeline(["This tutorial is extremely helpful.", "I don't like waiting for long installations."])
for res in results:
    print(res)
```

```
[transformers] No model was supplied, defaulted to distilbert/distilbert-base-uncased-finetuned-sst-2-english and revision 714eb0f.
Using a pipeline without specifying a model name and revision in production is not recommended.
config.json: 100% 629/629 [00:00<00:00, 62.8kB/s]
Loading weights: 100% 104/104 [00:00<00:00, 2705.20it/s]
tokenizer_config.json: 100% 48.0/48.0 [00:00<00:00, 4.60kB/s]
vocab.txt: 100% 232k/232k [00:00<00:00, 8.62MB/s]
{'label': 'POSITIVE', 'score': 0.9992333650588989}
{'label': 'NEGATIVE', 'score': 0.9677647352218628}
```

11. Load the BERT tokenizer and tokenize a sentence.

```
bert_tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
sentence = "Deep learning powers self-driving cars."
tokens = bert_tokenizer.tokenize(sentence)
print("Tokens:", tokens)
```

```
Tokens: ['deep', 'learning', 'powers', 'self', '-', 'driving', 'cars', '.']
```

12. Load a pre-trained BERT model and generate contextual embeddings.

```
config.json: 100% 570/570 [00:00<00:00, 45.7kB/s]
model.safetensors: reconstructing file: 100% 440MB / 440MB, 40.5MB/s
model.safetensors: downloading bytes: 415MB, 37.4MB/s
Loading weights: 100% 199/199 [00:00<00:00, 5310.60it/s]
[transformers] BertModel LOAD REPORT from: bert-base-uncased
Key                                         | Status     |  |
--------------------------------------------+------------+--+-
cls.predictions.transform.dense.bias        | UNEXPECTED |  |
cls.seq_relationship.bias                   | UNEXPECTED |  |
cls.predictions.transform.LayerNorm.weight  | UNEXPECTED |  |
cls.predictions.transform.dense.weight      | UNEXPECTED |  |
cls.predictions.transform.LayerNorm.bias    | UNEXPECTED |  |
cls.predictions.bias                        | UNEXPECTED |  |
cls.seq_relationship.weight                 | UNEXPECTED |  |

Notes:
- UNEXPECTED:    can be ignored when loading from different task/architecture; not ok if you expect identical arch.
Shape of embeddings: torch.Size([1, 8, 768])
```

13. Generate text using a pre-trained GPT-2 model.

```
Loading weights: 100% 148/148 [00:00<00:00, 3218.89it/s]
[transformers] Passing `generation_config` together with generation-related arguments=({'max_length', 'temperature', 'pad_token_id'}) is deprecated and will be removed in fut
[transformers] Both `max_new_tokens` (=256) and `max_length`(=40) seem to have been set. `max_new_tokens` will take precedence. Please refer to the documentation for more inf
In the future, robots will be able to do the things we've done before but we'll probably never have the tools we have now.

I am very excited about this. It's the first time we'll see humans become fully autonomous and use the same tools we have today.

Is it going to happen or will it be more complicated?

It's just that there's been a lot of excitement about it. It's a very exciting time for robotics. I think it will be very interesting for robots to be able to say "We have a

The best part about being a robot is that you can do things that you didn't think you could do before.

I'm optimistic. It's going to be a revolution that's for sure.

I've been a big proponent of this idea for a long time. I think it's going to change the way we think about the future. I think it's going to make it easier for us to do thin
```

14. Question Answering using a Pre-trained Transformer

```
from transformers import AutoModelForQuestionAnswering, AutoTokenizer
import torch

model_name = 'distilbert-base-cased-distilled-squad'
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForQuestionAnswering.from_pretrained(model_name)

context_text = "Google Colab is a hosted Jupyter notebook service that requires no setup to use."
question_text = "What is Google Colab?"

inputs = tokenizer(question_text, context_text, return_tensors="pt")
with torch.no_grad():
    outputs = model(**inputs)

answer_start = torch.argmax(outputs.start_logits)
answer_end = torch.argmax(outputs.end_logits) + 1
answer = tokenizer.convert_tokens_to_string(tokenizer.convert_ids_to_tokens(inputs["input_ids"][0][answer_start:answ

print({'answer': answer})
```

```
Loading weights: 100% 102/102 [00:00<00:00, 2911.18it/s]
{'answer': 'a hosted Jupyter notebook service'}
```

15. Tokenize a sentence using the BERT tokenizer and display the generated tokens and
token IDs.

```
tokens = bert_tokenizer.tokenize("Natural Language Processing with BERT!")
token_ids = bert_tokenizer.convert_tokens_to_ids(tokens)

for tok, tid in zip(tokens, token_ids):
    print(f"Token: {tok:<15} ID: {tid}")
```

```
Token: natural         ID: 3019
Token: language        ID: 2653
Token: processing      ID: 6364
Token: with            ID: 2007
Token: bert            ID: 14324
Token: !               ID: 999
```

16. Compare the tokenization outputs of BERT and GPT-2.

```
from transformers import GPT2Tokenizer

gpt2_tokenizer = GPT2Tokenizer.from_pretrained('gpt2')
sample_text = "Unbelievably pre-trained tokenizers operate differently."

print("BERT Tokens:", bert_tokenizer.tokenize(sample_text))
print("GPT-2 Tokens:", [gpt2_tokenizer.decode([i]) for i in gpt2_tokenizer.encode(sample_text)])
```

```
BERT Tokens: ['un', '##bel', '##ie', '##va', '##bly', 'pre', '-', 'trained', 'token', '##izer', '##s', 'operate', 'differently', '.']
GPT-2 Tokens: ['Un', 'bel', 'iev', 'ably', ' pre', '-', 'trained', ' token', 'izers', ' operate', ' differently', '.']
```

17. Generate contextual embeddings for a sentence using BERT.

```
with torch.no_grad():
    outputs_emb = bert_model(**inputs_emb)

# Mean pooling over token embeddings to represent the sentence
mean_embedding = torch.mean(outputs_emb.last_hidden_state, dim=1)
print("Contextual Sentence Embedding Shape:", mean_embedding.shape)
```

```
Loading weights: 100% ████████████████████ 199/199 [00:00<00:00, 3151.65it/s]
[transformers] BertModel LOAD REPORT from: bert-base-uncased
Key                                        | Status     |  |
-------------------------------------------+------------+--+-
cls.predictions.transform.dense.bias       | UNEXPECTED |  |
cls.seq_relationship.bias                  | UNEXPECTED |  |
cls.predictions.transform.LayerNorm.weight | UNEXPECTED |  |
cls.predictions.transform.dense.weight     | UNEXPECTED |  |
cls.predictions.transform.LayerNorm.bias   | UNEXPECTED |  |
cls.predictions.bias                       | UNEXPECTED |  |
cls.seq_relationship.weight                | UNEXPECTED |  |
```

18. Compare contextual embeddings generated for two semantically similar sentences.

```
bert_tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
bert_model = BertModel.from_pretrained('bert-base-uncased')

sentence_emb = "Hugging Face transformers provide great APIs."
inputs_emb = bert_tokenizer(sentence_emb, return_tensors="pt")
with torch.no_grad():
    outputs_emb = bert_model(**inputs_emb)

# Mean pooling over token embeddings to represent the sentence
mean_embedding = torch.mean(outputs_emb.last_hidden_state, dim=1)
print("Contextual Sentence Embedding Shape:", mean_embedding.shape)
```

```
Loading weights: 100% ████████████████████ 199/199 [00:00<00:00, 3151.65it/s]
[transformers] BertModel LOAD REPORT from: bert-base-uncased
Key                                        | Status     |  |
-------------------------------------------+------------+--+-
cls.predictions.transform.dense.bias       | UNEXPECTED |  |
cls.seq_relationship.bias                  | UNEXPECTED |  |
cls.predictions.transform.LayerNorm.weight | UNEXPECTED |  |
cls.predictions.transform.dense.weight     | UNEXPECTED |  |
cls.predictions.transform.LayerNorm.bias   | UNEXPECTED |  |
cls.predictions.bias                       | UNEXPECTED |  |
cls.seq_relationship.weight                | UNEXPECTED |  |

Notes:
```

19. Generate text from a prompt using the GPT-2 language model.

```
In this article, we'll learn how to learn Python while using Python. We will also show how to use Python in your daily life to learn Python.

What is the Python programming language?

Python is a programming language, a programming language that can be used and used safely with any computer, including PCs.

Why Python?

Python comes in two flavors:

Py

Python is a programming language that can be used in very many ways. It's a single-purpose language, which means that it can be used very much like any other programming lang

The main difference between Python and Python is that it's in a single language, whereas Python is a single-purpose language.

Python is a single-purpose language that can be used with any computer, including PCs. If you need to learn Python to learn how to program, you should learn Python as a singl

Python is an open-source software development framework. Python is a programming language that can be used with any computer.

How do I learn Python?

You can learn Python from
```

20. Generate and compare responses for multiple prompts using a pre-trained LLM.

```
Prompt 1: The key benefits of renewable energy are
Response: The key benefits of renewable energy are:

The ability to generate power for a wide range of applications, including the energy sector, transportation, infrastructure and other energy sectors.

The ability to provide a stable and affordable supply of renewable energy that meets industry needs and supports industry's economic growth.

The availability of low-carbon sources, such as solar and wind, that can be exported to markets that pay for their use.

The ability to offer a low-carbon supply of energy that meets industry needs and supports industry's economic growth.

The ability to provide a stable and affordable supply of clean energy that meets industry needs and supports industry's economic growth.

Solar PV installation can be carried out on the ground, with the installation of a solar panel or other solar wind turbine as a backup. The installation is carried out on the

When the wind is not blowing, the installation of a solar panel or other solar wind turbine is carried out on the ground with the installation of a solar panel or other solar

The installation of a solar panel or other solar wind turbine is carried out on the ground with the installation of a solar panel or other solar wind turbine.

Solar PV installations
--------------------------------------------------
Prompt 2: The potential risks of artificial general intelligence are
Response: The potential risks of artificial general intelligence are substantial. In particular, it is not possible to accurately predict the future, although this is importa

The real risk is that artificial intelligence will attempt to imitate the human brain, which is already becoming increasingly powerful. One of the most significant examples i
```